

Criando o *primeiro programa*

Do arquivo vazio ao "Olá, mundo!" no terminal. Nesta aula: o que é um programa, onde escrever o código, como rodar em Windows, Linux e macOS, o que o `print()` sabe fazer, como comentar o código e como ler as primeiras mensagens de erro, com a ajuda do agente do Cursor.

Arquivo .py

python3 ola.py

print()

sep e end

Comentários

Modo interativo

Lendo erros

Formação Claude Code 2026 · Baseado em docs.python.org (Tutorial, Built-in Functions: print) e cursor.com/docs, consultados em outubro de 2026 · Exemplos testados com Python 3.14 na pasta `exemplos/` · Saídas reais

O que é *um programa*

Um programa é uma **lista de instruções** que o computador executa **de cima para baixo**, uma linha de cada vez. Em Python, essas instruções ficam num arquivo de texto com a extensão `.py`.

VOCÊ ESCRIVE

`ola.py`



O PYTHON LÊ

o interpretador
executa linha a linha



RESULTADO

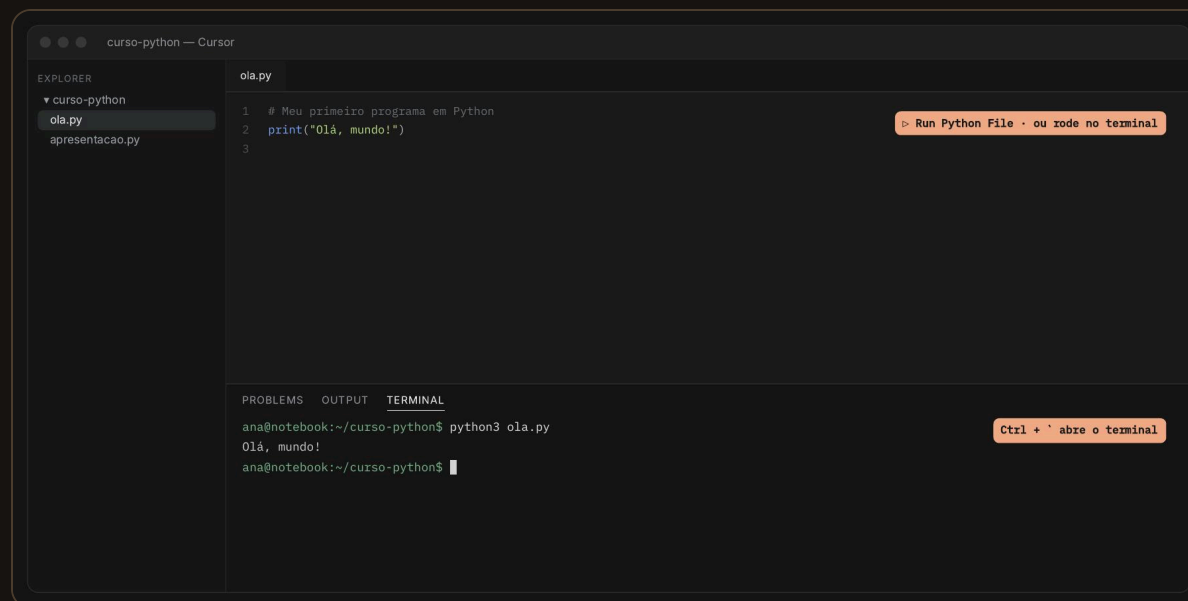
aparece no terminal

INTERPRETADOR

O **Python** que você instalou é um programa que lê o seu arquivo e executa cada instrução. Por isso se diz que Python é uma linguagem **interpretada**: não há um passo separado de compilação para você rodar.

Ainda não instalou? A instalação está no capítulo 2 do **Guia Completo Python** (aula 01) e no módulo 35 (Cursor). Confira com `python3 --version` (no Windows: `py --version`).

Escrevendo e rodando *no Cursor*



Arquivo no editor e saída no terminal · reprodução ilustrativa

1. Crie uma pasta, abra no Cursor e crie o arquivo `ola.py` .

2. Abra o terminal: `Ctrl` + ``` (crase).

3. Rode com `python3 ola.py` , ou pelo botão ▶ da extensão Python.

"Olá, *mun*do!"

ola.py

```
# Meu primeiro programa em Python  
print("Olá, mundo!")
```

SAÍDA

Olá, mundo!

- ✓ `print()` mostra na tela o que estiver entre os parênteses
- ✓ Texto vai entre aspas: `"..."` ou `'...'`
- ✓ A linha com `#` é um comentário: o Python ignora

SISTEMA

COMANDO PARA RODAR

Windows

`py ola.py` ou `python ola.py`

Linux

`python3 ola.py`

macOS

`python3 ola.py`

O terminal precisa estar **na pasta** do arquivo. O terminal do Cursor já abre na pasta do projeto.

De cima *para baixo*

apresentacao.py

```
# Um programa: instruções executadas de cima para baixo
print("Olá! Eu sou um programa em Python.")
print("Estou aprendendo a programar.")
print() # print vazio pula uma linha
print("Fim.")
```

SAÍDA

```
Olá! Eu sou um programa em Python.
Estou aprendendo a programar.

Fim.
```

- ✓ Cada `print()` termina com uma quebra de linha
- ✓ `print()` vazio imprime uma linha em branco
- ✓ A ordem da saída é a ordem do código

Vários valores, *sep* e *end*

print_varios.py

```
# Vários valores no mesmo print: o Python separa com espaço
print("Nome:", "Ana", "| Idade:", 29)

# sep: o que vai entre os valores
# end: o que vai no final (padrão: quebra de linha)
print("2026", "10", "05", sep="-")
print("Carregando", end="...")
print(" pronto!")

# Números e contas também podem ser impressos
print(2 + 3)
print("2 + 3")
```

SAÍDA

```
Nome: Ana | Idade: 29
2026-10-05
Carregando... pronto!
5
2 + 3
```

PARÂMETRO	PADRÃO	MUDA
sep	um espaço	O que vai entre os valores
end	quebra de linha	O que vai no final

Repare: `print(2 + 3)` mostra **5**; `print("2 + 3")` mostra o **texto**. Aspas fazem toda a diferença.

Notas *para quem lê*

comentarios.py

```
# Linha inteira de comentário: o Python ignora

print("Isto roda")  # comentário no fim da linha

# print("Isto NÃO roda: a linha está comentada")

"""
Texto entre três aspas no começo do arquivo
costuma servir de documentação (docstring).
"""
```

SAÍDA

```
Isto roda
```

- ✓ `#` comenta do símbolo até o fim da linha
- ✓ Comentário explica o **porquê**; o código já diz o quê
- ✓ Comentar uma linha é um jeito rápido de "desligá-la" para testar
- ✓ No Cursor, `Ctrl` + `/` comenta e descomenta a linha (`Cmd` + `/` no macOS)

O texto entre `"""` não é comentário de verdade: é uma string que o Python ignora por não ser usada. No início de arquivos e funções, ela vira **documentação** (docstring).

Testando linha a linha: *o REPL*

Digite `python3` (ou `py` no Windows) sem nome de arquivo e o Python abre o **modo interativo**, o REPL: você digita uma linha, ele executa e mostra o resultado na hora.

```
$ python3
>>> 2 + 3
5
>>> print("Olá!")
Olá!
>>> "Python" * 2
'PythonPython'
>>> exit()
```

- ✓ Ótimo para **experimentar** uma ideia rápida
- ✓ No REPL, o valor aparece sem `print()`; num arquivo, não
- ✓ Sair: `exit()` ou `Ctrl + D` (Linux/macOS), `Ctrl + Z` e Enter (Windows)
- ✓ O que você digita no REPL **se perde** ao sair: programas de verdade vão para arquivos

Desde o Python 3.13, o REPL tem cores, edição de várias linhas e histórico melhor (capítulo 21 do guia).

Os primeiros *erros*

erro_aspas.py · FALTOU FECHAR AS ASPAS

```
print("Olá, mundo!)
```

```
File "erro_aspas.py", line 1
  print("Olá, mundo!)
      ^
SyntaxError: unterminated string literal (detected at line 1)
```

erro_nome.py · PRINT COM P MAIÚSCULO

```
Print("Olá, mundo!")
```

```
Traceback (most recent call last):
  File "erro_nome.py", line 1, in <module>
    Print("Olá, mundo!")
    ^^^^^
NameError: name 'Print' is not defined. Did you mean: 'print'?
```

Python diferencia **maiúsculas de minúsculas**: `print` existe, `Print` não. E repare na sugestão do próprio Python: *Did you mean: 'print'?*

Lendo *uma mensagem de erro*

`erro_recuo.py` · ESPAÇO NO COMEÇO DA LINHA

```
print("primeira linha")
    print("segunda linha")
```

```
File "erro_recuo.py", line 2
    print("segunda linha")
IndentationError: unexpected indent
```

Em Python, o **recuo** (espaços no começo da linha) tem significado: ele marca blocos de código. Fora de um bloco, a linha começa na coluna 1.

PARTE DA MENSAGEM	O QUE DIZ
File "...", line 2	Arquivo e linha onde o Python percebeu o problema
A linha de código	O trecho exato
^^^^	Aponta a parte problemática
Última linha	Tipo do erro e explicação: comece por ela

Ler a **última linha primeiro** resolve a maioria dos erros de iniciante: ela diz o que aconteceu; o resto diz onde.

Aprendendo *com o Cursor*

MODO ASK · EXPLICAR SEM RESOLVER

Estou aprendendo Python. Rodei @erro_nome.py e apareceu NameError. Explique o que essa mensagem quer dizer, em linguagem simples, e me dê uma dica. NÃO corrija o arquivo: quero tentar sozinho.

DEPOIS DE CORRIGIR

Corrigi o arquivo. Está certo? Tem algum outro detalhe que um iniciante costuma errar aqui?

- ✓ Use o modo **Ask** (aula 03 do módulo 36): ele explica e não edita
- ✓ Peça **dica**, não a resposta: quem aprende é quem digita
- ✓ Cole o erro com `@Terminals` ou selecione o texto no terminal
- ✓ Desconfie da resposta e **rode de novo**: o terminal é o juiz final

Pedir para o agente fazer todos os exercícios é o jeito mais rápido de não aprender. Use-o como professor particular, não como colega que faz a prova.

Seu *cartão de visita*

Enunciado: crie `cartao.py` que imprima seu nome, sua profissão, e-mail e telefone na **mesma linha** separados por `" | "` (use `sep`) e, no fim, uma linha com 30 sinais de igual.

Uma resposta possível

```
# Exercício 1: cartão de visita
print("Ana Souza")
print("Desenvolvedora Python")
print("ana@exemplo.com", "(81) 99999-0000", sep=" | ")
print("=" * 30)
```

SAÍDA

```
Ana Souza
Desenvolvedora Python
ana@exemplo.com | (81) 99999-0000
=====
```

- ✓ `"=" * 30` repete o texto 30 vezes (mais na aula 05)
- ✓ Tente antes de olhar a resposta
- ✓ Desafio: imprima o cartão dentro de uma "moldura" de asteriscos

Teste o que *ficou*

1. Como rodar o arquivo ola.py no Linux ou macOS?

A) run ola.py

B) python3 ola.py

C) print ola.py

D) ola.py --python

B. No Windows: py ola.py.

2. O que `print("A", "B", sep="-")` mostra?

A) A B

B) A-B

C) AB

D) "A"-"B"

B. sep troca o espaço padrão.

3. `Print("oi")` dá qual erro?

A) `SyntaxError`

B) `NameError`

C) `IndentationError`

D) Nenhum

B. Python diferencia maiúsculas.

4. Por onde começar a ler uma mensagem de erro?

A) Pela primeira linha

B) Pela última linha: tipo e explicação

C) Pelo nome do arquivo

D) Não precisa ler

B. Depois, a linha indicada.

Primeiro programa *rodando*

Um programa é uma sequência de instruções num arquivo `.py`, executada de cima para baixo. Você rodou com `python3` (ou `py`), usou `print()` com `sep` e `end`, comentou o código, testou no REPL e leu os primeiros erros pela **última linha**.

`ola.py`

`print()`

`sep · end`

`#`

`REPL`

Última linha do erro

Próxima aula → 03 · Tipos de Dados

Fontes: docs.python.org/3/tutorial (Using the Python Interpreter, An Informal Introduction), docs.python.org/3/library/functions.html#print; cursor.com/docs (Ask mode, Terminal), consultados em outubro de 2026. Exemplos em `exemplos/`, executados com Python 3.14.4; as saídas foram geradas por eles.